Introduction to Programming (CSL100 / MAL400)

Name: Roll No:

Time 3 Hours Total marks 100

Instructions:

- Answer all questions
- Answers should be given in the space provided. Use space judiciously.
- The last page can be used for rough work.
- To answer the output of a given code snippet, provide traces of code execution on the right-hand side of the code.
- None of the codes have compiler errors, and header files are included by default.
- Write traces of code snippets. Only writing the output of the code will not get you any marks.
- Always provide justification for your answers
- Step marks may be given

Section 1: Q1 to Q10: Each code snippet question carries 2 marks. (2x10)

Q1

```
void modify(int *arr, int size)
{
   for (int i = 0; i < size; i++)
   {
      *(arr + i) = *(arr + i) * 2; // Each element in the array is multiplied by 2
   }
}
int main()
{
   int numbers[5] = {1, 2, 3, 4, 5}; // Array initialized with {1, 2, 3, 4, 5}
   modify(numbers, 5); // Array is modified to {2, 4, 6, 8, 10}
   for (int i = 0; i < 5; i++)
   {
      printf("%d ", numbers[i]); // Outputs: "2 4 6 8 10 "
   }
   return 0;
}
```

Outputs: "2 4 6 8 10 "

Q2

```
void modifyString(char *str)
{
   str[0] = 'H';  // Modify the first character to 'H'
   str[1] = 'e';  // Modify the second character to 'e'
   str[2] = 'l';  // Modify the third character to 'l'
   str[3] = 'l';  // Modify the fourth character to 'l'
   str[4] = 'o';  // Modify the fifth character to 'o'
}
int main()
{
   char str[] = "World"; // Initialize string as "World"
   printf("Before modification: %s\n", str); // Prints: Before modification: World
   modifyString(str); // Changes "World" to "Hello"
   printf("After modification: %s\n", str);  // Prints: After modification: Hello
   return 0;}
```

Output: Before modification World
After Modification Hello

Q3

```
int main()
{
   short a[3][2] = {3, 6, 9, 12, 15, 18};
// a[0] = {3, 6}, a[1] = {9, 12}, a[2] = {15, 18}
   printf("%d\n", *(a + 1)[1]);
// (a + 1): Pointer to the second row (a[1] = {9, 12})
   // *(a + 1): Dereference to access second row
   // *(a + 1)[1]: Access second element of second row (a[1][1] = 12)
   return 0;
}
```

Output: 12

Q4

```
struct Point {
   int x, y;
};
void updatePoint(struct Point *p)
{
   p->x += 10; // Increment x by 10
   p->y += 20; // Increment y by 20
}
int main()
{
   struct Point p = {5, 5};
 // Initialize struct p with x = 5, y = 5
   updatePoint(&p);
   // Pass the address of p to updatePoint
   // p->x becomes 5 + 10 = 15
   // p->y becomes 5 + 20 = 25
   printf("x = %d, y = %d\n", p.x, p.y);
   return 0;
}
```

Output: x = 15, y = 25

Q5

```
struct Node
{
   int data;
   struct Node* next;
};
void deleteFirst(struct Node** head)
{
   if (*head == NULL) return;  // Do nothing if list is empty
   struct Node* temp = *head;  // Store current head
   *head = (*head)->next;     // Move head to next node
   free(temp);   // Free old head
}
int main()
{
   struct Node* head = (struct Node*)malloc(sizeof(struct Node));
 head->data = 10;     // Set first node data
   head->next = (struct Node*)malloc(sizeof(struct Node));
   head->next->data = 20; // Set second data
   head->next->next = NULL; // Terminate
   deleteFirst(&head);  // Delete first node
   printf("%d", head->data);
   return 0;
}
```

Output: 20

Q6

```
void manipulateString(char *str) {
   str[0] = 'H';    // Change str[0] to 'H'
   str[1] = 'i';    // Change str[1] to 'i'
   str = "Hello, World!";      // Assign new string to str (local change)
   printf("Inside manipulateString: %s\n", str);  // Print modified str (local value)
}
int main() {
   char str[] = "Goodbye";   // Initialize str with "Goodbye"
   printf("Before manipulateString: %s\n", str);
   manipulateString(str);
   printf("After manipulateString: %s\n", str);   //only first two character will change and other will remain unchanged
   return 0;
}
```

Output: Before manipulateString: Goodbye
Inside manipulateString: Hello, World!
After manipulateString: Hiodby

```
Q7int recursiveFunction(int n)
{
 if (n == 0) // Base case: when n reaches 0
       return 1;
 return n * recursiveFunction(n - 1);
// Example recursiveFunction(5) = 5 *
recursiveFunction(4)
}
```

```
int main()
{
   int result = recursiveFunction(5);
   // Result: 5 * 4 * 3 * 2 * 1 = 120
   printf("Result: %d\n", result);
   return 0;
}
Output: Result: 120
```

```
Q8
void updateValues(int *a, int *b)
{
  *a += 5; // Add 5 to the value pointed by a
  *b += 10; // Add 10
}
void allocateMemoryAndUpdate(int **arr,
int size) {
   *arr = (int *)malloc(size * sizeof(int));
   for (int i = 0; i < size; i++) {
      (*arr)[i] = i * 2; // Initialize array
   }
}
int main() {
   int x = 3, y = 7;
   int *arr = NULL;
    updateValues(&x, &y);
// x becomes 3 + 5 = 8
// y becomes 7 + 10 = 17
```

```
   printf("After updateValues: x = %d, y =
%d\n", x, y);
   allocateMemoryAndUpdate(&arr, 5);
   //allocated array: [0, 2, 4, 6, 8]
   printf("Array elements after
allocateMemoryAndUpdate: ");
   for (int i = 0; i < 5; i++)
   {
      printf("%d ", arr[i]);
   }
   printf("\n");
   free(arr);
      return 0;
}
Output:After updateValues: x = 8, y = 17
Array elements after
allocateMemoryAndUpdate: 0 2 4 6 8
```

```
Q9
void modifyArray(int *arr)
{
   arr[0] = 10; // Update the first element
   arr[1] = 20; // Update the second
   arr[2] = 30; // Update the third element
   arr = arr + 2; // Change the local pointer
to point to the 3rd element
   arr[0] = 40; // Update the 3rd element
}
int main() {
   int arr[] = {1, 2, 3, 4, 5};
   modifyArray(arr);
```

```
   printf("Array after modifyArray: ");
   for (int i = 0; i < 5; i++)
  {
      printf("%d ", arr[i]);
   }
   printf("\n");
   return 0;
}

Output: Array after modifyArray: 10 20
40 4 5
```

```
Q10
void modifyArray(int arr[2][3])
{
 arr[0][0] = 10; //1st element of 1st row
arr[1][1] = 20;  //2nd element of  2nd row
```

```
}
int main()
{
   int arr[2][3] = {{1, 2, 3}, {4, 5, 6}};
```

```
    printf("Before modification:\n");
    for (int i = 0; i < 2; i++)
      {
        for (int j = 0; j < 3; j++)
        {
            printf("%d ", arr[i][j]);
        }
        printf("\n");
    }
    modifyArray(arr);
    printf("\nAfter modification:\n");
    for (int i = 0; i < 2; i++)
{
        for (int j = 0; j < 3; j++)
        {
            printf("%d ", arr[i][j]);
        }
        printf("\n");   }
        return 0;  }
```

Output: **Before modification:**
1 2 3
4 5 6

After modification:
10 2 3
4 20 6

Section 2: Q-11 to Q-18 carries 10 marks each for each question. (10x8)

Q-11 Write a command line C program to implement the following Linux command:
cat fin.txt > fout.txt
'Cat' fin.txt displays the contents of fin.txt in stdout
'>' copies the output of the left-hand side to the right-hand side file

```
int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s <input_file>
<output_file>\n", argv[0]);
        return 1;
    }

    FILE *fin = fopen(argv[1], "r");
    if (fin == NULL) {
        perror("Error opening input file");
        return 1;
    }

    FILE *fout = fopen(argv[2], "w");
    if (fout == NULL) {
        perror("Error opening output file");
        fclose(fin);
        return 1;
    }

    char line[1024];

    while (fscanf(fin, " %[^\n]", line) == 1)
{
        fprintf(fout, "%s\n", line);
        printf("%s\n", line);
    }

    fclose(fin);
    fclose(fout);

    printf("\nContents successfully
copied to %s\n", argv[2]);
    return 0;
}
```

Q-12) Consider the Homework 3 assignment grade management system: Add the following admin-only function to this software, which should calculate the average grade for each course across all students, providing insight into the overall performance in each course.

- After the user successfully logs in, if the UID is 0 (admin), they will have access to the following option:
  Calculate Course Average for All Students (Admin Only)
- This function will read the index.txt file, which contains user details in the format:

Content of Index.txt:

```
101,John Doe,Math:85,Science:90,History:75

103,Tom Brown,Math:92,Physics:80,Chemistry:70

102,Jane Smith,Math:78,Science:82,English:88
```

Function execution output:

```
Average grade for Math: 85.00
Average grade for Science: 86.00
Average grade for History: 75.00
Average grade for English: 88.00
Average grade for Physics: 80.00
Average grade for Chemistry: 70.00
```

```
/*Complete the function*/
void course_average() {
FILE *file = fopen("index.txt", "r");
    if (file == NULL) {
        printf("Error: Could not open index.txt\n");
        return;
    }
    char line[500];
    float course_sums[10] = {0};
    int course_counts[10] = {0};
    char course_names[10][50];
    int course_index = 0;
    while (fgets(line, sizeof(line), file)) {
        char *token = strtok(line, ",");
        token = strtok(NULL, ","); // Skip UID, and then get name
 // Process the courses and grades
        while ((token = strtok(NULL, ",")) != NULL) {
            char course[50];
            int grade;
            sscanf(token, "%[^:]:%d", course, &grade);
 // Search if the course is already in the list
            int found = 0;
            for (int i = 0; i < course_index; i++) {
                if (strcmp(course_names[i], course) == 0) {
                    course_sums[i] += grade;
                    course_counts[i]++;
                    found = 1;
                    break;
                }
            }
            if (!found) {
                strcpy(course_names[course_index], course);
                course_sums[course_index] = grade;
                course_counts[course_index] = 1;
                course_index++;
            }
        }
    }
        fclose(file)
        printf("Average grade for each course:\n");
        for (int i = 0; i < course_index; i++)
        {
            printf("Course: %s, Average Grade: %.2f\n", course_names[i], course_sums[i] / course_counts[i]);
            }}
```

Q-13) Write a program to delete the middle element of a singly linked list. If the list has an even number of nodes, delete the second middle node.

Example Cases:

- Input: 1 -> 2 -> 3 -> 4 -> 5
  Output: 1 -> 2 -> 4 -> 5
- Input: 1 -> 2 -> 3 -> 4
  Output: 1 -> 2 -> 4

```
struct Node {

   int data;

   struct Node* next;

};

/*Complete the function*/

struct Node* deleteMiddle(struct Node*
head) {
   // If the list is empty or has only one
node
   if (head == NULL || head->next == NULL)
{
      return NULL;
   }
   struct Node* slow = head;
   struct Node* fast = head;
   struct Node* prev = NULL;

   // Move the fast pointer 2 steps and the
slow pointer 1 step
   while (fast != NULL && fast->next !=
NULL) {
      fast = fast->next->next;  // Move fast
pointer by 2 steps
      prev = slow;   // Save the previous
node of slow
      slow = slow->next;       // Move slow
pointer by 1 step
   }

   // At this point, slow is pointing to the
middle node, and prev is pointing to the
node before slow
   prev->next = slow->next;  // Remove the
middle node

   // Free the memory of the deleted node
   free(slow);

   return head;

    }
```

Q-14) Write a C program that defines a structure for a book with fields for the title, author,name, and price. Create an array of books and implement the following functionalities:

1. A function to update the price of a specific book, identified by its title.
2. A function to display all books with a price above a given threshold.
3. A function to find the book with the highest price and display its details.

```c
typedef struct {
    char title[100];
    char author[100];
    float price;
} Book;

// Function to update the price of a specific book by title
void updatePrice(Book books[], int size, const char *title, float newPrice) {
    for (int i = 0; i < size; i++) {
        if (strcmp(books[i].title, title) == 0) {
            books[i].price = newPrice;
            printf("Price updated for '%s'.\n", title);
            return;
        }
    }
    printf("Book titled '%s' not found.\n", title);
}

// Function to display books with a price above a certain threshold
void displayBooksAbovePrice(Book books[], int size, float threshold) {
    printf("\nBooks with price above %.2f:\n", threshold);
    int found = 0;
    for (int i = 0; i < size; i++) {
        if (books[i].price > threshold) {
            printf("Title: %s, Author: %s, Price: %.2f\n", books[i].title, books[i].author, books[i].price);
            found = 1;
        }
    }
    if (!found) {
        printf("No books found above the given price.\n");
    }
}

// Function to find and display the most expensive book
void displayMostExpensiveBook(Book books[], int size) {
    int maxIndex = 0;
    for (int i = 1; i < size; i++) {
        if (books[i].price > books[maxIndex].price) {
            maxIndex = i;
        }
    }
    printf("\nThe most expensive book is:\n");
    printf("Title: %s, Author: %s, Price: %.2f\n", books[maxIndex].title, books[maxIndex].author, books[maxIndex].price);
}

int main() {
    // Initialize an array of books
    Book books[5] = {
        {"The Great Gatsby", "F. Scott Fitzgerald", 10.99},
        {"1984", "George Orwell", 14.99},
        {"To Kill a Mockingbird", "Harper Lee", 12.49},
        {"The Catcher in the Rye", "J.D. Salinger", 8.99},
        {"Moby Dick", "Herman Melville", 18.50}
    };

    int size = 5;

    // Display all books
    printf("Initial Book List:\n");
    for (int i = 0; i < size; i++) {

        printf("Title: %s, Author: %s, Price: %.2f\n", books[i].title, books[i].author, books[i].price);

    }


    // Update the price of a book
    updatePrice(books, size, "1984", 16.99);

    // Display books above a given price
    displayBooksAbovePrice(books, size, 12.00);

    // Display the most expensive book
    displayMostExpensiveBook(books, size);

    return 0;
}
```

Q-15) You are given an $n \times n$ 2D matrix representing an image. Rotate the image by 90 degrees (clockwise). You need to rotate the image in place, meaning you have to modify the input 2D matrix directly. Do not allocate another 2D matrix and perform the rotation.

Example:

Input:

matrix = [[5, 1, 9, 11],

[2, 4, 8, 10],

[13, 3, 6, 7],

[15, 14, 12, 16]]

Output:

[[15, 13, 2, 5],

[14, 3, 4, 1],

[12, 6, 8, 9],

[16, 7, 10, 11]]

```
void rotate(int matrix[][20], int n)
 {
// Step 1: Transpose the matrix
   for (int i = 0; i < n; i++) {
      for (int j = i; j < n; j++) {
         // Swap matrix[i][j] with matrix[j][i]
         int temp = matrix[i][j];
         matrix[i][j] = matrix[j][i];
         matrix[j][i] = temp;
      }
   }

   // Step 2: Reverse each row
   for (int i = 0; i < n; i++) {
      int start = 0, end = n - 1;
      while (start < end) {
         int temp = matrix[i][start];
         matrix[i][start] = matrix[i][end];
         matrix[i][end] = temp;
         start++;
         end--;
      }
   }
}

void printMatrix(int matrix[][20], int n)
{

for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++) {
         printf("%d ", matrix[i][j]);

      }
      printf("\n");
   }
}
int main() {
   int n = 3;
   int matrix[20][20] = {
      {1, 2, 3},
      {4, 5, 6},
      {7, 8, 9}
   };
   printf("Original Matrix:\n");
   printMatrix(matrix, n);
   rotate(matrix, n);
   printf("\nRotated Matrix:\n");
   printMatrix(matrix, n);
   return 0;
}
```

Q-16) Write a C program to check if a given string is a palindrome or not using pointers. You must not use any built-in string functions (e.g., strlen, strcmp, etc.). A palindrome is a string that reads the same backward or forward. Note that the check must be case-sensitive, meaning "Madam" is not considered a palindrome, but "madam" is. Complete the code given below to full fill the above task.

```
void isPalindrome(char* string)

{

char *start = string;
   char *end = string;

   // Move the end pointer to the last
character of the string
   while (*end != '\0') {
      end++;
   }
   end--;  // Now end points to the last
character

   // Compare characters from both ends
   while (start < end) {
      if (*start != *end) {
         printf("Not a palindrome\n");
         return;
      }
      start++;
      end--;
   }

   printf("Palindrome\n");
}

int main()

{

   char str[1000] ;

   printf(“enter string “);

   scanf(“%s”,str);

   isPalindrome(str);

   return 0;

}
```

Q-17) You are tasked with implementing a function that computes the square root of a given non-negative integer x. The function should return the square root y if x is a perfect square (i.e., $y^2=x$). If x is not a perfect square, the function should return -1.

Constraints:

1. You cannot use any built-in exponentiation functions (like `pow`, `sqrt`, or `**`).
2. The solution must be optimized to minimize the number of comparisons.

Input Example:
x=16x = 16x=16
Output:
4

Input Example:
x=20x = 20x=20
Output:
-1

```
int findPerfectSquareRoot(int x) {
if (x < 0) return -1; // Negative numbers do not have a real square root

   if (x == 0 || x == 1) return x; // Square root of 0 is 0, and square root of 1 is 1

   int left = 0, right = x, mid;

   // Perform binary search
   while (left <= right) {
      mid = (left + right) / 2;
      long long square = (long long)mid * mid;

      if (square == x) {
         return mid; // Found the exact square root
      } else if (square < x) {
         left = mid + 1; // Search in the higher half
      } else {
         right = mid - 1; // Search in the lower half
      }
   }

   return -1; // If no exact square root is found
}
int main() {
   printf("Square root of 16: %d\n", findPerfectSquareRoot(16)); // Output: 4
   printf("Square root of 20: %d\n", findPerfectSquareRoot(20)); // Output: -1
   printf("Square root of 1: %d\n", findPerfectSquareRoot(1));   // Output: 1
   printf("Square root of 0: %d\n", findPerfectSquareRoot(0));   // Output: 0
   return 0;
}
```

Q-18) You are given **n** jobs, each with a unique Job ID (e.g., A, B, C), a Deadline (the latest time by which the job can be completed, where each job takes exactly 1 time unit), and a Profit (the profit earned by completing the job within its deadline). Your task is to schedule the jobs such that the total profit is maximized, ensuring no job is scheduled beyond its deadline and only one job is performed at a time. Write a C program to read the number of jobs (n) and their details, schedule the jobs to maximize profit without using built-in sorting functions, and display the selected jobs, their sequence, and the total profit.

Example : Enter the number of jobs: 5

Enter job details (ID, Deadline, Profit):

A 2 100

B 1 50

C 2 10

D 1 20

E 3 80

Output: Scheduled Jobs: A E B

Total Profit: 230

```
typedef struct Job {
char id;
        int dead;
        int profit;
       } Job;
int compare(const void* a, const void* b)
{
        Job* temp1 = (Job*)a;
        Job* temp2 = (Job*)b;
        return (temp2->profit -
temp1->profit);
}
 int min(int num1, int num2)
{
        return (num1 > num2) ? num2 :
num1;
}
void printJobScheduling(Job arr[], int n)
{

// Sort all jobs according to decreasing
order of profit
    qsort(arr, n, sizeof(Job), compare);

    int result[n]; // To store result (Sequence
of jobs)
    bool slot[n]; // To keep track of free time
slots

    // Initialize all slots to be free
    for (int i = 0; i < n; i++)
        slot[i] = false;

    // Iterate through all given jobs
    for (int i = 0; i < n; i++) {

        // Find a free slot for this job (Note
that we start
        // from the last possible slot)
        for (int j = min(n, arr[i].dead) - 1; j >=
0; j--) {

            // Free slot found
            if (slot[j] == false) {
                result[j] = i; // Add this job to
result
                slot[j] = true; // Make this slot
occupied
                break;
            }
        }
    }

    // Print the result
    for (int i = 0; i < n; i++)
        if (slot[i])
            printf("%c ", arr[result[i]].id);
}

int main()
{
        Job arr[] = { { 'a', 2, 100 },
                { 'b', 1, 19 },
                { 'c', 2, 27 },
                { 'd', 1, 25 },
                { 'e', 3, 15 } };
        int n = sizeof(arr) / sizeof(arr[0]);
        printf("Following is maximum profit
sequence of jobs \n");
        printJobScheduling(arr, n);
        return 0;
}
```